Madrasa Management System Database Design and Architecture Review

Overview

This document describes the current database design, entity relationships, operational flow, and review findings for the Madrasa Management System. It is written to help future developers quickly understand how the system is structured, how the main modules interact, and which design decisions are intentional versus temporary compatibility choices.[cite:1]

The current stack uses Next.js App Router, TypeScript, Supabase for database access, and a custom JWT cookie session system rather than Supabase Auth as the main login flow.[cite:1] Route organization is split into role-based dashboard areas for `admin` and `lecturer`, which matches the current product structure of broad admin management and scoped lecturer access.[web:1][cite:1]

Application structure

The dashboard routes are grouped under `app/(dashboard)` with separate areas for `admin` and `lecturer`, and Next.js route groups do not affect the public URL structure.[web:1] The current dashboard tree includes admin sections for attendance, calendar, classes, departments, lecturers, students, subjects, and timetable, while lecturer currently has attendance and timetable views plus a main dashboard page.[cite:1]

Authentication currently uses custom JWT session cookies through helper functions such as `createSession()`, `getSession()`, and `destroySession()`, with middleware checking only whether a session cookie exists.[cite:1] Role protection is already implemented in layout files for admin and lecturer pages, while API-level role authorization is intentionally postponed for a later phase.[cite:1]

Core design approach

The schema follows a mostly relational design with separate tables for master data, operational scheduling, and student lifecycle tracking.[cite:1] At a high level, the system is built around departments, classes, lecturers, students, subjects, timetable entries, attendance records, and progression rules.[cite:1]

The strongest current design principle is incremental compatibility. Existing columns and relationships are preserved where the application already depends on them, and improvements are best made by tightening constraints, foreign keys, naming, and source-of-truth rules rather than by large structural rewrites.[cite:1]

Main entities

Area	Main tables	Purpose
Department structure	`departments`, `classes`, `subjects`	Organizes the academic hierarchy and subject ownership by department.[cite:1]
Lecturer domain	`lecturer`, `lecturer_language`, `lecturer_qualification`, `users`	Stores lecturer profile data, login linkage, and lecturer-related details.[cite:1]
Student lifecycle	`students`, `student_enrollments`, `passed_students`, `class_progression_map`	Stores personal details, current and historical enrollment, pass-out records, and progression rules.[cite:1]
Scheduling	`time_slots`, `timetable`	Represents day-specific class slots and assigned subject or lecturer combinations.[cite:1]
Operations	`attendance`	Stores daily attendance per student for a timetable entry and date.[cite:1]

Department model

The department module is intentionally simple. Admin users create a department by entering a name, type, and optionally an in-charge lecturer, with `is_active`, `created_at`, and `updated_at` supporting lifecycle state and auditing.[cite:1] The `type` field is constrained to `full-time`, `part-time`, or `special`, which matches the current business categories used in the UI.[cite:1]

The class model belongs directly to a department and stores class name, optional in-charge lecturer, default strength, and active status.[cite:1] Subjects are also department-owned, which means subject selection for timetable creation is expected to be department-scoped in the application.[cite:1]

Department relationships

- `classes.department_id -> departments.id` defines each class as a child of one department.[cite:1]
- `subjects.department_id -> departments.id` defines each subject as a child of one department.[cite:1]
- `departments.incharge_lecturer_id -> lecturer.lecturer_id` links the department in-charge to the lecturer internal key.[cite:1]
- `classes.incharge_lecturer_id -> lecturer.lecturer_id` links the class in-charge to the lecturer internal key.[cite:1]

Department business rules

Department deletion is intentionally treated as a hard, high-friction admin action rather than a casual workflow action.[cite:1] When a department is deleted, its related classes and subjects are intended to be removed as well, while inactive state is used in the UI for normal day-to-day deactivation and greying out.[cite:1]

Department review notes

This section aligns well with the business process, but it relies on careful delete behavior choices. `ON DELETE CASCADE` is appropriate for child entities that should disappear with the department, while `ON DELETE SET NULL` is appropriate for optional in-charge lecturer assignments that should survive lecturer unlinking.[web:11][cite:1]

Recommended improvements for this area include case-insensitive uniqueness on department names and on class or subject names within a department, plus a non-negative check on `classes.default_strength`.[web:53][web:27][cite:1]

Lecturer model

The lecturer module combines profile data, account linkage, and repeatable child records. The main `lecturer` table stores admission data, personal details, appointment details, madrasa details, signatures, remarks, and the account-facing visual name used for generating login credentials.[cite:1]

A lecturer account is created through the add-lecturer flow. At that point, a corresponding row is also created in the `users` table with role `lecturer`, and the resulting user UUID is stored as part of the lecturer-account relationship.[cite:1]

Lecturer detail tables

- `lecturer_language` stores one-to-many language records per lecturer.[cite:1]
- `lecturer_qualification` stores one-to-many qualification records per lecturer.[cite:1]
- Both child tables correctly use `ON DELETE CASCADE` because they have no meaning without the lecturer parent row.[cite:1]

Lecturer identity strategy

The lecturer design currently uses two identifiers with different purposes:[cite:1]

Identifier	Type	Current role
`lecturer.lecturer_id`	Integer primary key	Internal relational key used by lecturer child tables and in-charge relationships.[cite:1]
`lecturer.old_id` moving to `lecturer_uuid`	UUID unique column	External app-facing lecturer identity used by timetable, lecturer pages, and account linkage.[cite:1]

This is a valid dual-identifier model. A primary key does not have to be a UUID; PostgreSQL supports integer primary keys just as well, and a separate UUID column can be used as a stable public or application-facing identifier.[web:186][web:194]

Why the integer key should stay

Removing `lecturer.lecturer_id` now would break existing foreign-key relationships from `departments`, `classes`, `lecturer_language`, and `lecturer_qualification`, because those tables currently reference the integer primary key.[cite:1] Dropping a referenced primary key without migrating all dependent foreign keys first is a high-risk dependency change in PostgreSQL.[web:190][cite:1]

Why the UUID should remain the external lecturer identity

The application code shows that lecturer-facing pages consistently load lecturer data by taking `users.lecturer_id`, then querying the lecturer table by UUID, and then using that same UUID for timetable and lecturer-specific queries.[cite:1] The `timetable.lecturer_id` column also references the lecturer UUID identity rather than the integer primary key, and the lecturers list API returns the UUID as the value exposed to the UI.[cite:1]

Lecturer account linkage

The current effective account relationship is:

- `users.id` is the login account UUID.[cite:1]
- `users.lecturer_id` stores the lecturer-facing UUID.[cite:1]
- lecturer lookup occurs by matching `users.lecturer_id` to the lecturer UUID column.[cite:1]

This design works, but the column naming has been confusing because the UUID column was previously named `old_id`, even though it functions as the active external identifier.[cite:1] Renaming `old_id` to `lecturer_uuid` is therefore a naming cleanup, not a behavioral rewrite.[cite:1]

Lecturer code usage of UUID

The current lecturer codebase uses the UUID in all important lecturer-facing queries:[cite:1]

- dashboard stats use lecturer UUID to filter timetable rows.[cite:1]
- lecturer timetable page uses lecturer UUID to load assigned timetable entries.[cite:1]
- lecturer attendance page uses lecturer UUID to load only the lecturer's current lessons.[cite:1]
- lecturers list API exposes the lecturer UUID as the returned `id` field for consumers.[cite:1]

Lecturer review notes

The lecturer section is structurally workable. The biggest improvement is clarifying the naming and documenting the rule that `lecturer_id` is the internal database key while `lecturer_uuid` is the app-facing identity key.[cite:1]

Another high-priority improvement is password handling. The `users.password` value should be stored as a secure hash rather than plain text, even if the original generated credentials are

shown one time to an admin after creation.[web:81][cite:1]

Student model

The student area is the largest and most business-sensitive part of the system. It supports student creation, editing, pass-out handling, bulk promotion, and current enrollment lookup.[cite:1]

The current design intentionally separates personal details from progression history:[cite:1]

- `students` stores long-lived personal and identity information.[cite:1]
- `student_enrollments` stores class assignment and academic status over time.[cite:1]
- `passed_students` stores a final pass-out snapshot for reporting and archival convenience.[cite:1]

Student creation and status flow

A student is first inserted into `students`, and an automatic enrollment step is triggered through `trigger_auto_enroll_student` after insert.[cite:1] During later promotion cycles, new or updated rows in `student_enrollments` track current class placement, academic year, and progression status.[cite:1]

Student editing also allows a status change such as active, passed out, or inactive, and pass-out behavior writes to `passed_students` using the current student context.[cite:1]

Student progression model

`student_enrollments` is intended to represent the academic movement of a student across classes and years.[cite:1] Each row records student, class, department, academic year, whether it is current, enrollment or promotion timing, and a status such as `active`, `promoted`, `withdrawn`, or `passed`.[cite:1]

This is the correct table for storing current academic placement and historical progression. However, the surrounding schema still keeps overlapping academic-state columns in `students`, which creates drift risk if all code paths do not update them consistently.[cite:1]

Passed-out model

`passed_students` keeps a one-row-per-student record with the admission number, full name, qualification, final class, department, pass-out date, and remarks.[cite:1] This is useful because it preserves a reporting-friendly historical snapshot without requiring complicated joins over historical progression data.[cite:1]

Student duplication risks

The current design stores overlapping academic state in several places, including `students.madrasa_grade`, `students.department`, `students.department_id`, `students.is_active`, `students.is_passed`, `student_enrollments.status`, and `passed_students`.[cite:1] This means future developers must be very careful about which table is considered authoritative for each use case.[cite:1]

Student review notes

The safest current approach is to keep the existing columns for compatibility while formally treating `student_enrollments` as the primary source of truth for current academic placement and progression state.[cite:1] The `students` table should be treated mainly as the source for student identity and long-lived demographic data, while `passed_students` should be treated as the pass-out archive layer.[cite:1]

A key integrity gap is the current uniqueness rule in `student_enrollments`. The existing unique constraint on `(student_id, class_id, is_current)` does not guarantee only one current enrollment per student, because a student could still have multiple `is_current = true` rows if the class differs.[cite:1] A partial unique index on `student_id` where `is_current = true` is the more accurate rule for the current business logic.[web:112][cite:1]

Class progression map

`class_progression_map` is the configuration table that defines how promotion works inside each department.[cite:1] Each row maps a current class to its next class and also stores a progression order value.[cite:1]

Progression purpose

This table should be the source of truth for:

- determining the next class for a current class in a department,[cite:1]
- deciding when no next class exists and the student should be marked passed out,[cite:1]
- and maintaining the intended class sequence per department.[cite:1]

Progression review notes

The table structure is good and should remain central to promotion logic. The current uniqueness on `(department_id, current_class_id)` is correct because a class should not map to multiple next classes in the same department.[cite:1]

Recommended hardening includes a positive check on `progression_order` and possibly stronger validation that `next_class_id`, when present, belongs to the same department as `department_id`.[cite:1]

Timetable model

The timetable area is built from two linked concepts:[cite:1]

- `time_slots`, which define the available slots for a specific class and day,[cite:1]
- `timetable`, which assigns a subject and optionally a lecturer to a slot.[cite:1]

Time slots

A time slot belongs to one department, one class, and one day of the week, and it stores slot number, start time, and end time.[cite:1] The current unique constraint on `(department_id, class_id, day_of_week, slot_number)` correctly prevents duplicate slot numbers within the same class-day context.[cite:1]

Timetable entries

A timetable row links class, subject, time slot, optional lecturer UUID, day of week, validity dates, and active state.[cite:1] The intent is that a department is selected first, then a class from that department, then a day, then slots and subject or lecturer assignments for that day.[cite:1]

Lecturer linkage in timetable

The timetable table stores lecturer identity using the lecturer UUID field, not the integer lecturer primary key.[cite:1] This is one of the reasons the lecturer UUID column must remain in place as the app-facing lecturer identifier.[cite:1]

Timetable review notes

The scheduling design is sensible, but database integrity is weaker than the UI flow. The schema currently duplicates `day_of_week` in both `time_slots` and `timetable`, and there is no visible database rule ensuring they always match for a linked row.[cite:1]

Similarly, there is no direct database rule shown to ensure that a subject belongs to the same department as the selected class, or that a time slot selected by `time_slot_id` truly belongs to the timetable row's class.[cite:1] These relationships are likely being enforced in application logic today, but future developers should understand that the database is not fully protecting these business rules yet.[cite:1]

Recommended improvements include `CHECK (start_time < end_time)` on `time_slots`, a valid date range check on `timetable` such as `valid_to is null or valid_to >= valid_from`, and later consideration of overlap prevention for active timetable entries across time periods.[web:138] [web:131][cite:1]

Attendance model

Attendance is recorded by selecting a department, then a synced class, then a date, and then marking attendance for students in the relevant class context.[cite:1] The `attendance` table stores the timetable entry, student, date, status, the user who marked it, and timestamps for creation and marking.[cite:1]

Attendance strengths

The unique constraint on `(timetable_id, student_id, date)` is a strong and appropriate rule because it prevents duplicate attendance marking for the same student in the same lesson on the same day.[cite:1] The `marked_by` foreign key to `users.id` also provides clear accountability for who entered the attendance row.[cite:1]

Attendance review notes

Attendance itself is one of the cleaner parts of the schema, but it depends heavily on upstream timetable and student-enrollment correctness.[cite:1] A likely missing integrity improvement is an explicit foreign key from `attendance.student_id` to `students.id`, assuming the current data already satisfies that relationship.[web:118][cite:1]

There is also no visible database rule ensuring the student belongs to the class implied by the selected timetable row, so that enforcement currently depends on the lecturer or admin attendance workflow in application code.[cite:1]

Authentication and authorization notes

The system uses custom JWT cookie sessions rather than Supabase Auth as the main login system.[cite:1] Middleware currently checks only for the presence of a session cookie, while role-based UI protection is done in layout files for admin and lecturer areas.[cite:1]

This is acceptable as a current project phase, but future developers should understand that API-level role protection is not yet fully implemented and remains an identified gap.[cite:1] Lecturer page scoping is already applied so lecturers only see their own timetable and attendance-relevant data through lecturer UUID filtering.[cite:1]

There is also an older `/api/auth/callback` path using Supabase `exchangeCodeForSession`, which appears to be leftover from an earlier auth approach and should be reviewed carefully before the authentication layer is considered fully cleaned up.[cite:1]

Naming and compatibility decisions

Several naming choices in the current schema reflect project history rather than ideal final naming:[cite:1]

- `lecturer.old_id` is actually the active lecturer UUID and should be renamed to `lecturer_uuid` for clarity.[cite:1]
- `students.department` text and `students.department_id` overlap semantically and should be treated carefully until a later cleanup phase.[cite:1]
- status-like information exists in multiple places for students, so future code must use a clearly documented source-of-truth rule.[cite:1]

The safest modernization approach for this codebase is an incremental one: keep application-facing columns stable where they are already used, then strengthen foreign keys, uniqueness, checks, and naming without large rewrites.[cite:1]

Current relationship summary

Parent	Child / linked table	Relationship role
`departments`	`classes`	Department owns classes.[cite:1]
`departments`	`subjects`	Department owns subjects.[cite:1]
`departments`	`students`	Student can be linked to department.[cite:1]
`classes`	`student_enrollments`	Enrollment ties a student to a class.[cite:1]
`students`	`student_enrollments`	One student can have many enrollment records over time.[cite:1]
`students`	`passed_students`	Passed-out archive row references the original student.[cite:1]
`classes`	`class_progression_map`	Progression rules map class flow inside a department.[cite:1]
`time_slots`	`timetable`	Timetable entry uses a specific slot.[cite:1]
`timetable`	`attendance`	Attendance belongs to a specific timetable entry and date.[cite:1]
`users`	`attendance`	Marked-by user is recorded on attendance entries.[cite:1]
`users`	`lecturer`	Lecturer account linkage is done through lecturer UUID identity.[cite:1]

Source-of-truth guide

For future developers, these are the safest current source-of-truth rules:[cite:1]

Concern	Primary source	Notes
Department definition	`departments`	Master department record.[cite:1]
Class definition	`classes`	Master class record under department.[cite:1]
Subject definition	`subjects`	Department-owned subject definition.[cite:1]
Lecturer profile	`lecturer`	Main lecturer profile and public app identity UUID.[cite:1]
Lecturer login account	`users`	Authentication account row with role.[cite:1]
Student identity	`students`	Long-lived personal and admission details.[cite:1]
Student current academic placement	`student_enrollments`	Should be treated as authoritative for current class progression state.[cite:1]
Student pass-out archive	`passed_students`	Historical final snapshot after pass-out.[cite:1]
Promotion rule	`class_progression_map`	Controls next-class logic by department.[cite:1]
Day slot structure	`time_slots`	Defines allowed slot positions per class and day.[cite:1]
Lesson assignment	`timetable`	Ties subject and lecturer to class-day-slot.[cite:1]
Daily attendance	`attendance`	One row per timetable, student, and date.[cite:1]

Priority fixes

Change now

1. Rename `lecturer.old_id` to `lecturer_uuid` and update code references so naming matches actual usage.[cite:1]
2. Hash `users.password` values instead of storing raw passwords.[web:81][cite:1]
3. Add the missing integrity fixes already identified in master data, including case-insensitive uniqueness where required and non-negative checks where appropriate.[web:53][web:27][cite:1]
4. Add the missing attendance foreign key to `students.id` if existing data allows it.[web:118][cite:1]
5. Strengthen `student_enrollments` so only one current enrollment can exist per student.[web:112][cite:1]

Change soon

1. Add timetable and slot validation checks such as `start_time < end_time` and valid date range checks.[web:138][cite:1]
2. Clarify and document student status source-of-truth rules in code and developer docs.[cite:1]
3. Review `users.lecturer_id` and formally document that it stores the lecturer UUID rather than the integer lecturer primary key.[cite:1]
4. Review the old `/api/auth/callback` route and remove or isolate it if it is no longer part of the real auth flow.[cite:1]

Safe for later

1. Full removal or consolidation of duplicate student state columns, because the application currently depends on them.[cite:1]
2. Deeper timetable cross-table enforcement such as overlap prevention or composite consistency constraints, because these can be added after the current business flows are stabilized.[web:131][cite:1]
3. Reconsidering whether the internal lecturer integer key should ever be replaced; there is no urgent need as long as its role remains clearly documented.[web:186][cite:1]

Final architecture assessment

The system already has a solid real-world shape for a madrasa context: department and class hierarchy, lecturer profile management, student progression, timetable scheduling, and daily attendance are all represented in dedicated tables rather than flattened into one large structure.[cite:1] The main challenges are not missing modules, but inconsistent naming, duplicate state representation, and business rules that are currently enforced more by UI and app logic than by the database itself.[cite:1]

The best path forward is incremental hardening. The current codebase should not be rewritten from scratch; instead, it should be improved by clarifying identity strategy, tightening constraints, documenting source-of-truth rules, and gradually strengthening authorization and relational integrity while preserving the current app contract.[cite:1]